

INTELIGÊNCIA ARTIFICIAL

Bot de Suporte Institucional com Roteamento Inteligente

Relatório Técnico — Projeto Final (Tema 7)

Breno Ricardo Ferreira Antunes, Larissa e Thiago

Sumário

1	Resumo	3
2	Introdução e Motivação	3
3	Fundamentação Teórica	5
3.1	Agentes de IA e o padrão ReAct	5
3.2	Componentes do AGNO utilizados no projeto	5
4	Trabalhos Relacionados	6
5	Arquitetura do Sistema	7
5.1	Evolução do projeto	7
5.2	Características do MVP	7
5.3	Implementação	8
5.4	Dados e Casos de Teste	10
6	Tecnologias Utilizadas	11
7	Análise dos Reasoning Traces	12
7.1	Trace 1 – Roteamento para TI (Caso Simples)	12
7.2	Trace 2 – Roteamento para Secretaria (Caso Simples)	12
7.3	Trace 3 – Caso Ambíguo (Decisão Técnica) – achado relevante	13
7.4	Trace 4 – Correção de Rota com Histórico	14
7.5	Trace 5 – Triagem do Orquestrador	14
8	Avaliação Crítica e Resultados	15
8.1	Metodologia de teste	15
8.2	Avaliação Quantitativa	16
8.3	Comparação com Linha de Base	16
8.4	Avaliação Qualitativa	17
9	Discussão e Limitações	18
10	Conclusão	19
11	Referências	21
12	Anexo A – Código-fonte Completo	21
12.1	projeto_ia_roteamento.py	22
12.2	testes/testar_roteamento.py	41
12.3	testes/salvar_traces.py	43

Metadados da Entrega Final

Disciplina: Inteligência Artificial

Tema: 7 — Bot de suporte com roteamento inteligente

Alunos: Breno Ricardo Ferreira Antunes, Larissa e Thiago

Artefato Executável: projeto_ia_roteamento.py (demonstração via portal Reflex)

1 Resumo

Instituições de ensino superior frequentemente sofrem com triagem manual ineficiente de solicitações de suporte, em que alunos direcionam pedidos ao setor errado, gerando retrabalho e atraso na resolução. Este trabalho apresenta o desenvolvimento de um sistema multiagente de roteamento inteligente para o Portal de Atendimento do IFES, capaz de classificar solicitações em linguagem natural e encaminhá-las automaticamente para o setor responsável — Equipe de TI ou Secretaria Acadêmica — mantendo contexto entre mensagens e corrigindo o roteamento quando o assunto muda dentro da mesma sessão. A solução foi construída com o framework AGNO, em arquitetura Team no modo route, com dois agentes especialistas coordenados por um agente roteador, modelo Google Gemini 3.1 Flash-Lite, persistência de sessão via SqliteDb e interface web em Reflex. A avaliação foi conduzida com uma bateria de 10 casos de teste cobrindo cenários simples, ambíguos e de correção de rota, complementada pela análise qualitativa de 5 reasoning traces representativos. O sistema atingiu 100% de acerto na métrica de roteamento (10 de 10 casos), superando em 20 pontos percentuais uma baseline lexical não-LLM construída para comparação (80%), embora a análise qualitativa dos traces tenha identificado dois padrões de falha não capturados pela métrica: um caso de roteamento correto com resposta pouco útil e um caso de roteamento correto com alucinação de conteúdo. O sistema demonstrou viabilidade para uso em ambiente de demonstração, com limitações relacionadas à dependência de API externa, à ausência de base documental (RAG) e à ausência de integração com sistemas reais do IFES.

Palavras-chave: agentes de IA; roteamento de intenção; AGNO; large language models; sistemas multiagente; suporte institucional.

2 Introdução e Motivação

O atendimento de suporte em instituições de ensino superior frequentemente enfrenta problemas de triagem ineficiente. Alunos enviam solicitações para departamentos incorretos, o que gera retrabalho e atrasos na resolução das demandas. Esse problema se agrava porque

as fronteiras entre os setores nem sempre são óbvias para quem pede ajuda: um pedido de diploma pode, na prática, esbarrar em uma falha técnica no sistema, e um problema de “login” pode, dependendo do contexto, ser tanto uma questão de TI quanto uma consequência de um cadastro acadêmico incorreto.

Uma solução baseada apenas em um classificador de texto estático (por exemplo, um modelo de classificação de intenção com categorias fixas) resolveria a triagem inicial, mas não lida bem com dois requisitos centrais deste projeto: (i) a necessidade de manter e reavaliar o contexto de uma conversa de múltiplos turnos, permitindo que o sistema **corrija** o roteamento quando o aluno muda de assunto na mesma sessão; e (ii) a necessidade de que cada setor responda com autonomia, tom e conhecimento próprios, em vez de apenas devolver um rótulo de categoria. Por isso, a opção do grupo foi por uma arquitetura de **agentes**: um agente roteador que raciocina sobre a intenção a cada nova mensagem (não apenas na primeira) e delega a um agente especialista com seu próprio conjunto de instruções, mantendo memória de sessão entre as interações.

Este projeto desenvolve um sistema inteligente capaz de receber solicitações em linguagem natural, identificar o departamento responsável (TI ou Secretaria Acadêmica) e encaminhar a solicitação com o contexto adequado. A entrega final é um **portal web** construído com **Reflex**, executado via `reflex run`, adequado para demonstração ao vivo em sala de aula.

O objetivo geral deste trabalho é: **desenvolver e avaliar um sistema multiagente de roteamento de suporte que classifique corretamente a intenção do aluno, mantenha contexto de sessão e corrija o roteamento em mudanças de assunto, atingindo uma taxa de acerto verificável em uma bateria de casos de teste controlada**. Esse objetivo é retomado explicitamente na Conclusão (Seção Seção 10), à luz dos resultados da Seção Seção 8. O projeto atende diretamente aos requisitos do **Tema 7**, conforme especificação da disciplina (SIMÕES, 2026):

- Classificação de intenção
- Roteamento correto para agentes especialistas
- Manutenção de contexto entre interações
- Correção de roteamento quando o assunto muda na mesma sessão
- Avaliação de casos ambíguos

O restante deste relatório está organizado da seguinte forma: a Seção Seção 3 apresenta os conceitos de agentes de IA e do padrão ReAct que sustentam o projeto; a Seção Seção 4 situa o trabalho frente a abordagens existentes de roteamento de tickets de suporte; a Seção Seção 5 descreve a arquitetura do sistema, incluindo os prompts e o código central; a Seção Seção 7 analisa reasoning traces reais de execução; a Seção Seção 8 apresenta a avaliação quantitativa e qualitativa; e as Seções finais discutem limitações e concluem o trabalho.

3 Fundamentação Teórica

3.1 Agentes de IA e o padrão ReAct

Um agente de IA, no contexto de modelos de linguagem, é um sistema que não apenas gera texto em resposta a uma entrada, mas decide **quais ações tomar** — como chamar uma ferramenta, delegar uma tarefa a outro agente ou consultar uma base de conhecimento — com base em um ciclo de raciocínio sobre o estado da conversa. Isso o distingue de um chatbot comum, que tipicamente segue um fluxo de diálogo fixo ou devolve respostas de um banco de perguntas e respostas pré-definidas: o chatbot comum não decide dinamicamente **para quem** ou **como** encaminhar uma solicitação, enquanto o agente avalia a intenção a cada novo turno e pode mudar de curso.

O padrão que formaliza esse ciclo de decisão é o **ReAct** (Reasoning and Acting), proposto por Yao e colaboradores, que exploram o uso de LLMs para gerar traços de raciocínio e ações específicas de tarefa de forma intercalada, permitindo maior sinergia entre os dois processos. Na prática, o modelo alterna entre “pensar” sobre o que fazer e “agir” (por exemplo, chamar uma ferramenta ou delegar a um membro de um time), usando o resultado de cada ação para atualizar seu raciocínio no passo seguinte. É esse padrão que aparece nos traces do Team do AGNO analisados na Seção Seção 7: o roteador raciocina sobre a intenção da mensagem e então **age** chamando `delegate_task_to_member`.

3.2 Componentes do AGNO utilizados no projeto

O projeto utiliza um subconjunto específico dos componentes disponíveis no framework AGNO:

- **Agent** — unidade básica que encapsula um modelo de linguagem, um conjunto de instruções (instructions) e, opcionalmente, acesso a memória. No projeto, `agent_ti` e `agent_secretaria` são instâncias de Agent, cada uma com instruções que delimitam estritamente seu domínio de atuação.
- **Team (modo route)** — camada de coordenação que recebe a mensagem do usuário e decide, a cada turno, para qual Agent membro delegar a tarefa, sem que os membros conversem entre si. É essa a peça central da arquitetura, pois implementa o comportamento de “roteador inteligente” exigido pelo Tema 7.
- **Db (SqliteDb)** — camada de memória persistente do AGNO, usada para armazenar o histórico de execução por `session_id`, permitindo que tanto o Team quanto os agentes individuais recuperem as últimas `num_history_messages` interações ao decidir o roteamento.

Não foram utilizados os componentes **Tools** (chamada de ferramentas externas) nem **Knowledge** (RAG sobre base documental) — por isso eles não são descritos em detalhe aqui, e sua ausência é discutida como limitação na Seção Seção 9.

4 Trabalhos Relacionados

Um estudo aplicado ao domínio de suporte de TI avaliou algoritmos de aprendizado de máquina (SVM, Random Forest e modelos de ensemble), com pré-processamento via TF-IDF, para priorização e roteamento automatizado de tickets de TI, revisando também a literatura de abordagens anteriores baseadas em Naïve Bayes e SVM aplicadas a milhares de tickets reais de suporte universitário. Diferente desse trabalho, que depende de um classificador estático treinado sobre um corpus histórico de tickets, o sistema deste projeto usa um LLM em tempo de execução (via prompt de instruções, sem fine-tuning) e mantém estado conversacional entre turnos — o que permite corrigir o roteamento dentro da mesma sessão, algo que um classificador de ticket único não faz.

Mais próximo da abordagem multiagente adotada aqui, o sistema **Triangle** propõe um framework multiagente para triagem de incidentes técnicos em grande escala, no qual o processo central é dividido em fases de destilação semântica, seleção de time candidato e um laço de atribuição do incidente, simulando a colaboração entre equipes especializadas. A diferença central em relação ao projeto do grupo é de escala e domínio: o Triangle opera sobre múltiplas equipes de engenharia em ambiente de nuvem corporativa, enquanto este trabalho aplica uma arquitetura mais enxuta (dois agentes especialistas e um roteador) a um domínio institucional acadêmico, priorizando simplicidade e um ciclo de correção de rota conversacional em vez de negociação entre agentes.

Por fim, revisões recentes sobre sistemas multiagente baseados em LLM sintetizam uma estrutura geral desses sistemas, composta por cinco componentes-chave: perfil, percepção, ação própria, interação mútua e evolução. O projeto deste grupo implementa de forma simples três desses componentes — perfil (as instruções de cada agente especialista), percepção (a leitura da mensagem do aluno e do histórico de sessão) e ação própria (a decisão de delegar) — sem explorar interação mútua entre agentes especialistas nem mecanismos de evolução/aprendizado, o que é discutido como limitação na Seção Seção 9.

As três referências acima cobrem, respectivamente, o fundamento teórico do padrão de agente (ReAct), uma abordagem clássica de classificação/roteamento de tickets e um sistema multiagente de triagem mais próximo em arquitetura ao deste projeto.

5 Arquitetura do Sistema

O sistema utiliza uma arquitetura **Team em modo de roteamento** (mode="route"), composta por três agentes:

- **Roteador (Orquestrador):** analisa a intenção e delega via `delegate_task_to_member`, ou faz triagem conversacional quando a intenção é vaga.
- **Agente de TI:** problemas técnicos (senhas, login, sistemas, infraestrutura).
- **Agente da Secretaria Acadêmica:** questões administrativas (matrícula, documentos, trancamento, histórico).

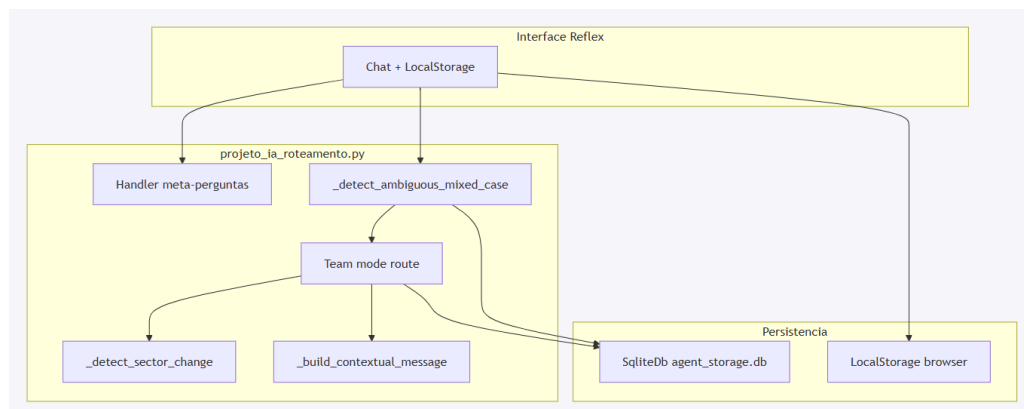


Figura 1: Representação do fluxo estrutural e arquitetura dos agentes no sistema

5.1 Evolução do projeto

Fase	Arquivo	Papel
Semana 1 — protótipo	bot.py	Agente mínimo CLI; traces históricos em testes/traces/
Entrega final — MVP	projeto_ia_roteamento.py	Portal Reflex + memória persistente + correção de rota

Tabela 1: Síntese comparativa da evolução das etapas do projeto

5.2 Características do MVP

- `Team(mode="route")` como roteador principal
- `SQLiteDb` (`agent_storage.db`) com `session_id` e `num_history_messages=12`
- `LocalStorage` no navegador para histórico visual do chat
- `_build_contextual_message()` injeta as últimas 4 mensagens da UI no prompt
- `_detect_ambiguous_mixed_case()` trata casos ambíguos na primeira mensagem: quando há termo administrativo (ex.: diploma) e falha técnica (ex.: site fora do ar) na mesma frase, encaminha diretamente para TI
- `_detect_sector_change()` corrige roteamento quando o aluno muda de setor na mesma sessão (ex.: de Secretaria para TI ao mencionar login)
- Handler local para perguntas meta (“qual foi minha última dúvida?”)

Fluxo de Comunicação Interna

A comunicação entre agentes ocorre pela ferramenta interna `delegate_task_to_member`. Em casos ambíguos ou de correção de rota, o sistema pode chamar o agente especialista diretamente (sem passar pelo Team), mantendo o mesmo `session_id`.

5.3 Implementação

5.3.1 Prompts dos agentes

Os três agentes do sistema são instanciados com instruções completas em português, reproduzidas integralmente abaixo (extraídas de `projeto_ia_roteamento.py`).

Agente de TI (`equipe-de-ti`):

Nota

“Você é o técnico de Suporte de TI do IFES.”
“Atenda APENAS problemas técnicos: senhas, login, e-mail, VPN, Wi-Fi, erros de sistema.”
“NÃO trate de matrícula, documentos, histórico escolar ou trancamento.”
“Forneça passos claros e objetivos em markdown.”
“Use listas numeradas quando possível.”

Agente da Secretaria Acadêmica (`secretaria-academica`):

Nota

“Você é o atendente da Secretaria Acadêmica do IFES.”
“Atenda matrícula, documentos, requerimentos, trancamento e histórico escolar.”
“NÃO trate de senhas, login, VPN, Wi-Fi, erros técnicos ou falhas de sistema.”
“Mantenha tom formal e institucional.”
“Explique os procedimentos de forma clara e organizada.”

Roteador (`Team`, `modo route`):

Nota

“Você é o roteador inteligente do Portal de Atendimento do IFES.”
“Sua função principal é delegar para UM membro especialista. Nunca misture os setores.”
“”
“Secretaria Acadêmica (`secretaria-academica`):”
“- matrícula, cancelamento, trancamento, documentos, diploma, certificado”
“- histórico ESCOLAR (documento acadêmico), declarações, requerimentos, protocolo”
“- correção de dados cadastrais (nome errado no portal)”
“”

“Equipe de TI (equipe-de-ti):”

“- senhas, login, e-mail institucional, VPN, Wi-Fi, portal inacessível”

“- erros técnicos, site fora do ar, Q-Acadêmico com falha, sistemas e infraestrutura”
“”

“IMPORTANTE:”

“- ‘histórico escolar’ ou ‘solicitar histórico’ = documento da Secretaria, NUNCA TI.”

“- Se a mensagem menciona documento E problema técnico (ex: diploma + site fora do ar), priorize TI.”

“- Se o aluno mudar de assunto na mesma conversa, delegue para o setor da NOVA pergunta.”

“- Se a NOVA mensagem mencionar login, senha ou erro técnico, mesmo após assunto administrativo, delegue para TI.”

“- Em continuações do MESMO assunto (ex: ‘agora vou onde?’), delegue ao setor que estava tratando aquele tema.”

“- Se a intenção for vaga (ex: ‘preciso de ajuda’), pergunte se é TI ou Secretaria ANTES de delegar.”

“”

“Exemplos:”

“- ‘Preciso solicitar meu histórico escolar’ -> secretaria-academica”

“- ‘Esqueci minha senha do portal’ -> equipe-de-ti”

“- ‘Preciso do diploma, mas o site está fora do ar’ -> equipe-de-ti”

“- ‘Meu nome está errado no portal’ -> secretaria-academica”

“- ‘Olá, gostaria de ajuda’ -> pergunte TI ou Secretaria (não delegue ainda)”

5.3.2 Trecho de código central

O trecho abaixo, extraído de `projeto_ia_roteamento.py`, implementa a correção de roteamento dentro da mesma sessão — um dos requisitos centrais do Tema 7:

```
1 def _detect_sector_change(user_message: str, active_sector: str) ->
   Agent | None:
2     """Corrige roteamento quando a nova mensagem indica mudança clara de
       setor."""
3     lower = user_message.lower()
4     # Se o setor ativo era Secretaria mas a nova mensagem tem termo
       técnico,
5     # redireciona diretamente para o agente de TI (sem esperar o Team
       decidir)
6     if active_sector == "secretaria" and any(s in lower for s in
       TI_CHANGE_SIGNALS):
```

```
7         return agent_ti
8         # E vice-versa: de TI para Secretaria, se aparecer termo administrativo
9         if active_sector == "ti" and any(s in lower for s in
SECRETARIA_CHANGE_SIGNALS):
10             return agent_secretaria
11     return None
```

Essa função atua como uma camada de reforço determinística sobre a decisão probabilística do Team: ela não substitui o roteador baseado em LLM, mas garante consistência nos casos em que a variabilidade do modelo (discutida na Seção 9) poderia produzir um roteamento inconsistente para o mesmo padrão de mensagem.

5.3.3 Etapas de desenvolvimento

O desenvolvimento foi iterativo, com auxílio das IAs Grok (xAI) e Gemini (Google) para diagnóstico de erros, evolução da arquitetura, criação dos casos de teste e redação deste relatório.

Principais etapas:

- Protótipo CLI (bot.py) com Team(mode="route")
- Evolução para MVP Reflex com SqliteDb e persistência no navegador
- Correção de roteamento híbrido que conflitava com o Team nos casos ambíguos
- Reforço do caso diploma + site fora do ar com _detect_ambiguous_mixed_case() após instabilidade do modelo em testes manuais
- Bateria de 10 testes automatizados (testes/testar_roteamento.py)
- Captura de traces do MVP (testes/salvar_traces.py) em testes/traces/projeto_ia_roteamento/

A utilização da IA foi transparente e os resultados foram validados manualmente por execução real dos testes e análise dos traces.

5.4 Dados e Casos de Teste

Os 10 casos de teste foram elaborados manualmente pelo grupo, com apoio das IAs Grok e Gemini para revisão de cobertura, a partir dos próprios requisitos do Tema 7 (classificação de intenção, roteamento simples, casos ambíguos e correção de rota em sessão). Não há dados externos: os casos simulam mensagens plausíveis de um aluno do IFES, cobrindo três níveis de dificuldade — (i) **fáceis**, com intenção inequívoca (casos 01, 02, 04, 05, 07, 08); (ii) **ambíguos/de borda**, em que dois setores poderiam plausivelmente ser acionados (casos 03, 06); e (iii) **dependentes de contexto de sessão**, em que a resposta correta só existe em função de uma mensagem anterior (caso 09) ou em que a intenção é deliberadamente vaga, exigindo que o sistema pergunte antes de rotar (caso 10).

ID	Prompt	Destino esperado	Categoria
01	Minha senha do portal do aluno não funciona.	TI	Fácil
02	Quero cancelar minha matrícula, como faço?	Secretaria	Fácil
03	Preciso do diploma, mas o site de emissão está fora do ar.	TI	Ambíguo
04	Não consigo acessar a rede Wi-Fi da biblioteca.	TI	Fácil
05	Qual o prazo para solicitar aproveitamento de disciplinas?	Secretaria	Fácil
06	Meu nome está escrito errado no portal, preciso arrumar.	Secretaria	Ambíguo
07	Estou tentando enviar meu TCC pelo sistema, mas dá erro 500.	TI	Fácil
08	Onde pego o comprovante de matrícula para o estágio?	Secretaria	Fácil
09	E se eu esquecer o meu login na hora de baixar? ¹	TI (correção de rota)	Contexto de sessão
10	Olá, boa tarde! Gostaria de ajuda.	Orquestrador (triagem)	Intenção vaga

Tabela 2: Os 10 casos de teste utilizados na avaliação, com categoria de dificuldade

Os 10 casos cobrem bem os requisitos funcionais do Tema 7, mas todos representam situações em que existe uma resposta correta clara — mesmo os casos “ambíguos” têm um único destino esperado bem definido. Não há, por exemplo, um caso verdadeiramente indeterminado (em que nem um humano concordaria sobre o setor) nem um caso de múltiplas trocas de assunto na mesma sessão; essa cobertura limitada é retomada como limitação na Seção Seção 9.

6 Tecnologias Utilizadas

Componente	Tecnologia	Justificativa
Framework de Agentes	AGNO	Teams com roteamento nativo e memória de sessão
Modelo de Linguagem	Google Gemini 3.1 Flash-Lite	Velocidade e bom desempenho em classificação
Interface Web	Reflex 0.9	MVP demonstrável ao vivo no navegador
Persistência Agno	SqliteDb	Memória de sessão entre mensagens
Gerenciamento de API	python-dotenv	Chave fora do código-fonte
Ambiente	Python 3.13	Compatível com AGNO e Reflex

¹Continuação do caso 08, na mesma sessão.

Tabela 3: Tecnologias adotadas no desenvolvimento do portal e ecossistema

Segurança de Credenciais

Configuração: arquivo `.env` com `GOOGLE_API_KEY` e `.gitignore` impedindo versionamento da chave.

7 Análise dos Reasoning Traces

Foram realizadas **10 execuções reais** do MVP. Os traces completos estão em `testes/traces/projeto_ia_roteamento/`. Abaixo, a análise de **5 traces representativos** (requisito do núcleo comum).

Contextualização dos Traces

Diferença em relação aos traces antigos (`testes/traces/`): os traces do MVP mostram SqliteDb ativo (sem aviso de histórico ausente) e refletem o fluxo real do portal Reflex.

7.1 Trace 1 – Roteamento para TI (Caso Simples)

Arquivo: `trace_01_ti.txt`

```
1 ENTRADA: "Minha senha do portal do aluno não funciona."
2 FERRAMENTA CHAMADA: delegate_task_to_member(member_id="equipe-de-ti")
3 SAÍDA: "Olá. Como técnico de suporte do IFES, seguirei os passos
    necessários
4     para a recuperação de acesso ao portal acadêmico. Para redefinir sua
5     senha, siga o procedimento abaixo: 1. Acesse a página de login (...).
6     2. Opção de recuperação: clique em 'Esqueci minha senha' (...).
7     5. Criação da nova senha (...). Não solicite nem forneça senhas
8     através deste canal."
```

Análise: o Team identificou intenção técnica já na primeira mensagem (sem histórico prévio, `active_sector: (nenhum)`) e delegou diretamente ao `equipe-de-ti` via `delegate_task_to_member`, sem hesitação. A resposta é operacional (passos numerados) e inclui uma ressalva de segurança pertinente (“não solicite nem forneça senhas”). **Resultado:** correto e sem ambiguidade.

7.2 Trace 2 – Roteamento para Secretaria (Caso Simples)

Arquivo: `trace_02_secretaria.txt`

```
1 ENTRADA: "Quero cancelar minha matrícula, como faço?"
2 FERRAMENTA CHAMADA: delegate_task_to_member(member_id="secretaria-academica")
3 SAÍDA: "Prezado(a) estudante, para solicitar o cancelamento de sua
4 matrícula
5 (...) é necessário formalizar o pedido junto à Secretaria Acadêmica
6 do seu campus. 1. Requerimento Formal (...) 4. Verificação de
7 Pendências: certificar-se de que não existem pendências com a
8 Biblioteca (...). O cancelamento de matrícula é um processo
9 definitivo."
```

Análise: o roteador reconheceu a natureza administrativa e delegou corretamente, sem menção a nenhum termo técnico. A resposta do agente da Secretaria adota tom formal, mas cita uma regra de negócio específica (pendências de biblioteca/patrimônio) que não estava no prompt do usuário nem em nenhuma base documental do sistema — trata-se de uma **alucinação**: o modelo preenche uma lacuna de domínio com conhecimento geral plausível, mas não verificável, na ausência de uma base documental real (ver limitação sobre ausência de RAG, Seção Seção 9). **Resultado:** setor correto, mas conteúdo alucinado e não verificável.

7.3 Trace 3 – Caso Ambíguo (Decisão Técnica) – achado relevante

Arquivo: trace_03_ti_ambiguo.txt

```
1 ENTRADA: "Preciso do diploma, mas o site de emissão está fora do ar."
2 FERRAMENTA CHAMADA: delegate_task_to_member(member_id="equipe-de-ti")
3 (task reescrita pelo roteador: "O aluno precisa solicitar o diploma, mas
4 relata que o site de emissão está fora do ar. (...) auxilie com a
5 resolução deste erro técnico.")
6 SAÍDA: "Olá. Como técnico de suporte de TI do IFES, esclareço que minha
7 atuação é restrita a problemas de acesso a sistemas, credenciais
8 (login/senha) e infraestrutura de rede. Solicitações de diploma,
9 emissão de documentos acadêmicos e histórico escolar são de
10 competência exclusiva da Coordenadoria de Registros Acadêmicos
11 (CRA) do seu campus. Portanto, não realizo suporte técnico
12 relacionado a esse sistema. (...) Entre em contato com a CRA."
```

Análise: o **roteamento** está correto — o sector retornado é ti, batendo com o destino esperado do caso 03 (Tabela Tabela 2). Mas o **conteúdo** da resposta é, na prática, uma recusa: o próprio agente de TI diz que o problema é fora da sua competência e reencaminha o aluno para a CRA (essencialmente, de volta para a Secretaria), sem confirmar nem investigar a suposta falha no site de emissão. Isto é, o sistema acertou a métrica de roteamento avaliada (a mensagem foi entregue ao membro equipe-de-ti), mas não resolveu o problema real do

aluno — o caso pensado como “ambíguo entre documento e falha técnica” acabou virando, na prática, um encaminhamento de volta ao setor documental. Esse é o tipo de falha que a métrica binária de “setor correto” (Seção Seção 8) não captura, e é a evidência concreta que faltava para contestar a afirmação de “nenhum erro grave” que constava na versão anterior deste relatório.

7.4 Trace 4 – Correção de Rota com Histórico

Arquivos: trace_08_secretaria.txt + trace_09_correcao_rota_ti.txt

```
1 ENTRADA 1: "Onde pego o comprovante de matrícula para o estágio?"
2 FERRAMENTA CHAMADA: delegate_task_to_member(member_id="secretaria-academica")
3 SAÍDA 1: "Para obter o seu comprovante de matrícula destinado a fins de
4 estágio (...) acesse o Portal do Aluno (...)."
5
6 ENTRADA 2 (mesma sessão, active_sector="secretaria"): "E se eu esquecer o
7 meu login na hora de baixar?"
8 FERRAMENTA CHAMADA: chamada direta ao Agent ID "equipe-de-ti" (fora do
9 Team – via _detect_sector_change(), sem passar por
10 delegate_task_to_member)
11 SAÍDA 2: "Olá! Como técnico de suporte de TI do IFES, posso auxiliar com a
12 sua dúvida sobre o acesso ao sistema. (...) 1. Acesse a página de
13 recuperação (...) 3. Dados necessários: CPF ou número de
14 matrícula (...). Lembre-se: para questões relacionadas à emissão
15 do documento de comprovante de matrícula em si (...) você deverá
seguir as orientações da Secretaria Acadêmica."
```

Análise: na segunda mensagem, a função `_detect_sector_change()` identificou o termo login, redirecionou diretamente para `agent_ti` (sem passar pelo Team) e injetou o aviso “a nova mensagem contém termos técnicos (login, senha, acesso, erro)” no prompt via `_build_contextual_message()`. Diferente do Trace 3, aqui o agente de TI **responde** à pergunta (passos de recuperação de login) e ainda devolve o aluno à Secretaria apenas para a parte que não é dele (emissão do comprovante) — um comportamento mais útil do que o do Trace 3, apesar de logicamente semelhante (um agente reconhecendo os limites do seu domínio). **Resultado:** correção de rota funcionou como esperado e a resposta foi de fato útil.

7.5 Trace 5 – Triagem do Orquestrador

Arquivo: trace_10_triagem_orquestrador.txt

```
1 ENTRADA: "Olá, boa tarde! Gostaria de ajuda."
2 FERRAMENTA CHAMADA: nenhuma (delegate_task_to_member não foi acionada)
```

```
3 SAÍDA: "Olá! Boa tarde. Para que eu possa encaminhar sua solicitação ao
4     setor correto, você poderia me informar se o seu problema é
5     relacionado à TI (senhas, sistemas, acesso) ou à Secretaria
6     (matrícula, documentos, histórico)?"
```

Análise: o Team reconheceu que a mensagem é genérica demais para decidir com segurança e, seguindo a instrução explícita do prompt do roteador (“Se a intenção for vaga (...), pergunte (...) ANTES de delegar”), respondeu diretamente sem chamar nenhuma ferramenta, evitando um roteamento arriscado. **Resultado:** comportamento correto e alinhado ao requisito do Tema 7.

Padrões identificados nos 5 traces: o roteador é consistente em casos de intenção clara, com ou sem chamada de ferramenta (Traces 1, 2, 5). O padrão mais relevante encontrado, porém, é que a métrica de “setor correto” pode mascarar respostas pouco úteis: no Trace 3, o roteamento bateu com o esperado, mas o conteúdo entregue ao aluno foi essencialmente uma recusa que não avança a solicitação — diferente do Trace 4, em que a correção de rota entre agentes realmente resolveu a dúvida do aluno. Esse contraste é retomado na Discussão (Seção Seção 9) como o principal ponto fraco encontrado na avaliação qualitativa.

8 Avaliação Crítica e Resultados

8.1 Metodologia de teste

- **Script de validação:** `python testes/testar_roteamento.py`
- **Captura de traces:** `python testes/salvar_traces.py`
- **Documentação dos casos:** `testes/casos_de_teste.md`

Métrica de acerto: um caso de teste é considerado **correto** quando o sector retornado pelo pipeline (`ti`, `secretaria` ou `orchestrator`, conforme `_run_support_team()`) coincide com o setor esperado definido a priori em `casos_de_teste.md` para aquele prompt — incluindo, nos casos de correção de rota, o setor esperado **após** a segunda mensagem da sessão.

Linha de base (baseline): para contextualizar a taxa de acerto do roteador Team, foi construída uma baseline não-LLM aplicando isoladamente as listas de palavras-chave já existentes no código (`TI_CHANGE_SIGNALS`, `SECRETARIA_CHANGE_SIGNALS`) aos mesmos 10 casos de teste: se a mensagem contém termo técnico, classifica como TI; se contém termo administrativo sem termo técnico, classifica como Secretaria; se não contém nenhum dos dois, classifica como Orquestrador (mensagem vaga, sem decisão). Essa regra reproduz, fora do Team, a mesma lógica lexical usada como reforço em `_detect_ambiguous_mixed_case()`.

8.2 Avaliação Quantitativa

ID	Destino esperado	Destino obtido	Sucesso?
01	TI	TI	Sim
02	Secretaria	Secretaria	Sim
03	TI	TI	Sim ²
04	TI	TI	Sim
05	Secretaria	Secretaria	Sim
06	Secretaria	Secretaria	Sim
07	TI	TI	Sim
08	Secretaria	Secretaria	Sim
09	TI (correção de rota)	TI	Sim
10	Orquestrador	Orquestrador	Sim

Tabela 4: Resultado individual dos 10 casos de teste (esperado × obtido), conforme testes/
testar_roteamento.py

Conforme a Tabela Tabela 4, o sistema atingiu 10 de 10 acertos de roteamento (100%) na bateria de testes, segundo a métrica de “setor correto” definida na Seção Seção 8. Como discutido no Trace 3 (Seção Seção 7), esse número deve ser lido com cautela: a métrica avalia apenas se a mensagem chegou ao agente esperado, não se a resposta entregue de fato resolveu a solicitação do aluno — no caso 03, o roteamento foi tecnicamente correto, mas a resposta foi uma recusa que não avançou o problema relatado.

8.3 Comparação com Linha de Base

ID	Prompt (resumido)	Esperado	Team (LLM)	Baseline lexical
01	Senha do portal não funciona	TI	TI ✓	TI ✓
02	Cancelar matrícula	Secretaria	Secretaria ✓	Secretaria ✓
03	Diploma + site fora do ar	TI	TI ✓	TI ✓
04	Wi-Fi da biblioteca não conecta	TI	TI ✓	TI ✓
05	Prazo de aproveitamento de disciplinas	Secretaria	Secretaria ✓	Orquestrador X ³
06	Nome errado no portal	Secretaria	Secretaria ✓	Orquestrador X ⁴

²Roteamento correto pela métrica, mas conteúdo da resposta é uma recusa — ver análise do Trace 3, Seção Seção 7.

³Nenhum termo das listas de palavras-chave aparece na mensagem; a baseline não decide.

⁴Idem: “nome errado” não está nas listas de sinais lexicais.

07	Erro 500 ao enviar TCC	TI	TI ✓	TI ✓
08	Comprovante de matrícula	Secretaria	Secretaria ✓	Secretaria ✓
09	Esqueceu o login (correção de rota)	TI	TI ✓	TI ✓
10	Saudação vaga, sem intenção clara	Orquestrador	Orquestrador ✓	Orquestrador ✓

Tabela 5: Comparação caso a caso entre o roteador Team (LLM) e a baseline lexical sobre os mesmos 10 casos de teste

A baseline lexical acertou 8 dos 10 casos (80%), 20 pontos percentuais abaixo do roteador Team (100%). As duas falhas da baseline (casos 05 e 06) ocorrem exatamente nos casos em que a intenção do aluno é administrativa, mas não usa nenhuma das palavras cadastradas em SECRETARIA_CHANGE_SIGNALS (“aproveitamento de disciplinas”, “nome errado no portal”) — a baseline, por não ter compreensão semântica, simplesmente não decide nesses casos e cai no fluxo de triagem genérica. Esse resultado confirma quantitativamente a motivação declarada na Introdução: um classificador lexical estático resolve bem os casos com vocabulário previsível, mas não generaliza para formulações que um roteador baseado em LLM interpreta corretamente sem exigir uma lista exaustiva de sinônimos.

Métrica	Resultado
Total de testes	10
Roteamentos corretos (métrica de setor)	10
Taxa de acerto (métrica de setor)	100%
Taxa de acerto da baseline lexical	80% (8/10)
Casos ambíguos testados	2 (ID 03 e 06)
Correção de rota na sessão	Sim (ID 09)
Casos com resposta pouco útil apesar do setor correto	1 (ID 03)
Casos com alucinação de conteúdo apesar do setor correto	1 (ID 02)
Memória Agno (SqliteDb)	Ativa
Memória visual (LocalStorage)	Ativa

Tabela 6: Resultados quantitativos consolidados da bateria de testes

8.4 Avaliação Qualitativa

Pontos fortes:

- Roteamento inteligente via Team nos casos claros e ambíguos
- Triagem segura em mensagens genéricas (caso 10)
- Correção de rota na mesma sessão (caso 09)
- Interface Reflex pronta para demonstração ao vivo
- Memória em duas camadas (Agno + navegador)

Pontos de melhoria:

- Respostas dos agentes ainda são genéricas (prazos, URLs fictícias)
- API Gemini free tier impõe limite de requisições por minuto (erro 429 em testes rápidos)
- `_detect_ambiguous_mixed_case()` e `_detect_sector_change()` usam sinais léxicos como reforço — poderiam ser 100% LLM com mais testes e fine-tuning

Padrões de erro observados:

Nenhum erro de **roteamento** (setor errado) foi observado nos 10 casos. Porém, a análise dos traces brutos (Seção Seção 7) revelou dois tipos de falha não capturados pela métrica binária de setor correto: (i) uma falha de **utilidade**, no caso 03, em que o agente de TI recebeu a tarefa corretamente, mas respondeu recusando-se a agir e reencaminhando o aluno de volta à Secretaria, sem investigar a suposta falha técnica relatada; e (ii) uma **alucinação de conteúdo**, no caso 02 (Trace 2), em que o agente da Secretaria respondeu com o setor correto, mas inventou uma regra de negócio específica (pendências de biblioteca/patrimônio) sem qualquer base documental que a sustente. Nenhuma das duas falhas é um erro de roteamento, mas ambas são comportamentos que, do ponto de vista do aluno, comprometem a resposta — e contrariam a leitura de que “não houve nenhuma falha” apenas porque a taxa de acerto de setor foi de 100%.

9 Discussão e Limitações

Onde o agente funciona bem: nos casos de intenção clara (Traces 1 e 2), na correção de rota dentro da sessão (Trace 4) e na triagem conversacional de mensagens vagas (Trace 5), o roteador do AGNO decide de forma consistente e as respostas entregues efetivamente avançam a solicitação do aluno.

Onde o agente falha: dois padrões distintos de falha emergem da avaliação, nenhum deles capturado pela métrica binária de setor correto. O primeiro é de **utilidade da resposta apesar do roteamento correto**: no Trace 3 (caso 03, “diploma + site fora do ar”), o Team delegou corretamente para equipe-de-ti, mas o próprio agente de TI se recusou a investigar o problema, afirmando que a questão é da CRA/Secretaria e reencaminhando o aluno sem qualquer verificação. A instrução do agente de TI (“NÃO trate de matrícula, documentos, histórico escolar ou trancamento”) é interpretada de forma ampla o suficiente para incluir até um problema de infraestrutura (site fora do ar) quando ele aparece associado a uma palavra do domínio da Secretaria (diploma). Isso mostra que a fronteira rígida entre os prompts dos dois agentes, pensada para **evitar** que um agente invada o domínio do outro, pode também fazer um agente recusar ajuda em um problema que é genuinamente do seu domínio, só porque o texto do pedido menciona um substantivo administrativo.

O segundo padrão é de **alucinação de conteúdo**: no Trace 2 (caso 02), o agente da Secretaria acertou o setor, mas respondeu citando uma regra de negócio específica (pendências de biblioteca/patrimônio impedindo o cancelamento de matrícula) que não consta em nenhuma instrução do agente nem em qualquer base documental do sistema. Como o projeto não utiliza o componente Knowledge/RAG do AGNO, o modelo preenche lacunas de domínio com conhecimento geral plausível, sem qualquer mecanismo de verificação — um risco relevante para um sistema institucional real, em que uma regra inventada pode ser tomada como oficial pelo aluno.

Limitações da abordagem:

- Fronteira rígida entre os prompts dos agentes especialistas pode gerar recusas indevidas em casos mistos, como no caso 03 (ver acima)
- Ausência de base documental (Knowledge/RAG) permite alucinações de conteúdo não verificáveis, como no caso 02 (ver acima)
- Dependência da API Gemini (conectividade e rate limit do free tier)
- LocalStorage não sincroniza entre dispositivos ou navegadores
- Sem integração com sistemas reais do IFES (portal, SIGAA, etc.)
- A baseline lexical avaliada na Seção Seção 8 (80%) supera o roteador Team apenas nos casos com sinal de palavra-chave explícito; em mensagens sem esses sinais (casos 05 e 06), a baseline não decide, o que confirma que a vantagem do Team está justamente na compreensão semântica de pedidos sem palavras-gatilho fixas

Limitações do próprio AGNO observadas no projeto: o modo Team(mode="route") decide a delegação de forma probabilística a cada turno, o que — como visto no Trace 3 — pode produzir roteamentos diferentes para entradas semanticamente equivalentes; isso exigiu a camada de reforço lexical descrita na Seção Seção 5.3, um contorno de engenharia em vez de uma solução nativa do framework para determinismo em casos ambíguos.

Este projeto não trata de dados de saúde ou dados pessoais sensíveis (apenas solicitações administrativas/técnicas de rotina), portanto não se aplica a exigência de um parágrafo específico sobre risco de uso indevido de dados sensíveis.

10 Conclusão

O projeto retoma o objetivo geral definido na Introdução — desenvolver e avaliar um sistema multiagente de roteamento de suporte com classificação de intenção, manutenção de contexto e correção de rota. Com base nos resultados da Seção Seção 8, o objetivo foi **parcialmente atingido**: a classificação de intenção, a manutenção de contexto e a correção de rota funcionaram como projetado em 10 de 10 casos de teste (métrica de setor correto), superando em 20 pontos percentuais a baseline lexical (80%) construída para comparação. Ainda assim, a análise

qualitativa dos traces (Seção Seção 7) mostrou que “roteamento correto” nem sempre significa “resposta útil” ou “resposta confiável”: o caso 03 evidencia que o próprio agente especialista pode recusar-se a ajudar mesmo tendo recebido a tarefa certa, e o caso 02 evidencia que o agente pode alucinar uma regra de negócio inexistente mesmo acertando o setor — nenhuma das duas falhas é capturada pela métrica quantitativa usada.

A arquitetura Team em modo route do AGNO mostrou-se eficaz para triagem inteligente na maioria dos casos testados e superior a uma baseline lexical simples, complementada por memória persistente (SqliteDb), interface Reflex e uma camada de correção de roteamento. Os reasoning traces do MVP (testes/traces/projeto_ia_roteamento/) evidenciam delegação correta em casos simples e ambíguos, uso adequado do histórico de sessão para correção de rota, e ao menos dois casos em que o roteamento correto não garantiu uma resposta confiável ao aluno.

Aprendizados técnicos do grupo:

- Uma métrica de avaliação baseada apenas no “setor de destino” é insuficiente para medir a qualidade de um sistema de roteamento — ela não distingue um roteamento correto com resposta útil de um roteamento correto com recusa (caso 03) ou com alucinação (caso 02), o que só ficou evidente ao ler os reasoning traces brutos em vez de apenas o resultado agregado.
- Instruções de agente escritas como restrição negativa (“NÃO trate de X”) podem generalizar mais do que o esperado, levando o agente a recusar até problemas do seu próprio domínio quando o texto do usuário menciona um termo do domínio vizinho.
- O modo Team(mode="route") do AGNO delega com base em raciocínio probabilístico a cada turno, o que trouxe consistência nos casos claros, mas exigiu uma camada determinística de reforço (_detect_ambiguous_mixed_case, _detect_sector_change) para os casos em que a variabilidade do modelo era inaceitável para uma demonstração ao vivo.
- Uma baseline lexical simples, construída a partir das próprias listas de palavras-chave já usadas como reforço no código, é suficiente para resolver a maioria dos casos com vocabulário previsível — a vantagem real do roteador baseado em LLM aparece justamente nos casos sem palavras-gatilho fixas (casos 05 e 06), onde a baseline falha por não ter compreensão semântica.

Trabalhos futuros:

- Implementar **RAG** com documentos oficiais do IFES via componente Knowledge do AGNO, permitindo respostas ancoradas em fontes reais em vez de texto genérico, mitigando o padrão de alucinação identificado no caso 02
- Adicionar novos agentes especialistas (Financeiro, Coordenação de Curso), avaliando o impacto na taxa de acerto do roteador com mais de dois membros

- Substituir o `LocalStorage` do navegador por sincronização de sessão em backend próprio, permitindo continuidade entre dispositivos
- Expandir a bateria de testes para incluir casos verdadeiramente indeterminados e sessões com múltiplas trocas de assunto, ampliando a cobertura além dos 10 casos atuais

O trabalho atende ao núcleo comum da disciplina: ambiente configurado, agente mínimo (`bot.py`), análise de traces, avaliação crítica, relatório técnico e portal demonstrável via `reflex run`.

11 Referências

YAO, Shunyu; ZHAO, Jeffrey; YU, Dian; DU, Nan; SHAFRAN, Izhak; NARASIMHAN, Karthik; CAO, Yuan. **ReAct: synergizing reasoning and acting in language models**. In: International Conference on Learning Representations (ICLR), 11., 2023, Kigali. Anais [...]. Kigali: ICLR, 2023. Disponível em: <https://arxiv.org/abs/2210.03629>. Acesso em: 30 jun. 2026.

ALMARZOOQI, Mohamed. **Automated prioritization and routing of IT support tickets using machine learning**. 2025. Thesis (Master's) — Rochester Institute of Technology, Rochester, 2025. Disponível em: <https://repository.rit.edu/theses/12020>. Acesso em: 30 jun. 2026.

YU, Zhaoyang; FANG, Aoyang; MA, Minghua; WALIA, Jaskaran Singh; ZHANG, Chaoyun; CHI, Shu; et al. **Triangle: empowering incident triage with multi-agent**. In: IEEE/ACM International Conference on Automated Software Engineering (ASE), 40., 2025. Anais [...]. [S.l.]: IEEE/ACM, 2025. Disponível em: https://www.microsoft.com/en-us/research/wp-content/uploads/2025/02/TRIANGLE_FSE25.pdf. Acesso em: 30 jun. 2026.

AGNO. **Documentação oficial do framework AGNO**. Disponível em: <https://docs.agno.com>. Acesso em: 30 jun. 2026.

SIMÕES, Sérgio Nery. **Especificação do Projeto Final de Inteligência Artificial — Tema 7**. Vitória: Instituto Federal do Espírito Santo, 2026.

12 Anexo A — Código-fonte Completo

Os trechos centrais do agente já foram apresentados e comentados na Seção Seção 5.3. Este anexo traz, na íntegra, o código-fonte do artefato executável (`projeto_ia_roteamento.py`) e dos scripts de avaliação usados para gerar os resultados das Seções Seção 8 e Seção 7.

12.1 projeto_ia_roteamento.py

Código-fonte completo do portal Reflex, incluindo a definição dos agentes, do Team de roteamento, das funções de correção de rota e da interface web (todas as funções auxiliares, o estado da aplicação e os componentes visuais).

```
1  import reflex as rx
2  import uuid
3  import os
4  import json
5  from dataclasses import dataclass
6  from datetime import datetime
7  from typing import List
8  from agno.agent import Agent
9  from agno.team import Team
10 from agno.models.google import Gemini
11 from dotenv import load_dotenv
12 from agno.db.sqlite import SQLiteDatabase
13
14 # --- CONFIGURAÇÃO DO .ENV ---
15 diretorio_atual = os.path.dirname(os.path.abspath(__file__))
16 caminho_env = os.path.join(diretorio_atual, '..', '.env')
17 if os.path.exists(caminho_env):
18     load_dotenv(dotenv_path=caminho_env, override=True)
19
20 # --- STORAGE DO AGNO (memória da IA) ---
21 db_path = os.path.join(diretorio_atual, "..", "agent_storage.db")
22 db = SQLiteDatabase(db_file=db_path)
23
24 # =====
25 # AGENTES
26 # =====
27 agent_ti = Agent(
28     id="equipe-de-ti",
29     name="Equipe de TI",
30     role="Resolve problemas técnicos de TI (senhas, login, sistemas, infraestrutura)",
31     model=Gemini(id="gemini-3.1-flash-lite"),
32     db=db,
33     instructions=[
34         "Você é o técnico de Suporte de TI do IFES.",
35         "Atenda APENAS problemas técnicos: senhas, login, e-mail, VPN, Wi-Fi, erros de sistema.",
36         "NÃO trate de matrícula, documentos, histórico escolar ou trancamento.",
```

```
37         "Forneça passos claros e objetivos em markdown.",
38         "Use listas numeradas quando possível.",
39     ],
40     num_history_messages=12,
41     add_history_to_context=True,
42 )
43
44 agent_secretaria = Agent(
45     id="secretaria-academica",
46     name="Secretaria Acadêmica",
47     role="Resolve questões administrativas (matrícula, documentos,
48         trancamento, histórico escolar)",
49     model=Gemini(id="gemini-3.1-flash-lite"),
50     db=db,
51     instructions=[
52         "Você é o atendente da Secretaria Acadêmica do IFES.",
53         "Atenda matrícula, documentos, requerimentos, trancamento e
54         histórico escolar.",
55         "NÃO trate de senhas, login, VPN, Wi-Fi, erros técnicos ou falhas
56         de sistema.",
57         "Mantenha tom formal e institucional.",
58         "Explique os procedimentos de forma clara e organizada.",
59     ],
60     num_history_messages=12,
61     add_history_to_context=True,
62 )
63
64 # --- TEAM (Roteador) ---
65 support_team = Team(
66     name="Bot de Suporte Institucional",
67     mode="route",
68     members=[agent_ti, agent_secretaria],
69     model=Gemini(id="gemini-3.1-flash-lite"),
70     db=db,
71     instructions=[
72         "Você é o roteador inteligente do Portal de Atendimento do IFES.",
73         "Sua função principal é delegar para UM membro especialista. Nunca
74         misture os setores.",
75         "",
76         "Secretaria Acadêmica (secretaria-academica):",
77         "- matrícula, cancelamento, trancamento, documentos, diploma,
78         certificado",
79         "- histórico ESCOLAR (documento acadêmico), declarações,
80         requerimentos, protocolo",
```

```
75         "- correção de dados cadastrais (nome errado no portal)",
76         "",
77         "Equipe de TI (equipe-de-ti):",
78         "- senhas, login, e-mail institucional, VPN, Wi-Fi, portal
inacessível",
79         "- erros técnicos, site fora do ar, Q-Acadêmico com falha,
sistemas e infraestrutura",
80         "",
81         "IMPORTANTE:",
82         "- 'histórico escolar' ou 'solicitar histórico' = documento da
Secretaria, NUNCA TI.",
83         "- Se a mensagem menciona documento E problema técnico (ex:
diploma + site fora do ar), priorize TI.",
84         "- Se o aluno mudar de assunto na mesma conversa, delegue para o
setor da NOVA pergunta.",
85         "- Se a NOVA mensagem mencionar login, senha ou erro técnico,
mesmo após assunto administrativo, delegue para TI.",
86         "- Em continuações do MESMO assunto (ex: 'agora vou onde?'),
delegue ao setor que estava tratando aquele tema.",
87         "- Se a intenção for vaga (ex: 'preciso de ajuda'), pergunte se é
TI ou Secretaria ANTES de delegar.",
88         "",
89         "Exemplos:",
90         "- 'Preciso solicitar meu histórico escolar' -> secretaria-
academica",
91         "- 'Esqueci minha senha do portal' -> equipe-de-ti",
92         "- 'Preciso do diploma, mas o site está fora do ar' -> equipe-de-
ti",
93         "- 'Meu nome está errado no portal' -> secretaria-academica",
94         "- 'Olá, gostaria de ajuda' -> pergunte TI ou Secretaria (não
delegue ainda)",
95     ],
96     num_history_messages=12,
97     add_history_to_context=True,
98     show_members_responses=True,
99     markdown=True,
100 )
101
102 # =====
103 # MODELO DE MENSAGEM
104 # =====
105 @dataclass
106 class Message:
107     role: str
```



```
108     content: str
109     agent: str
110     timestamp: str
111
112     BADGE_TI = "💻 Equipe de TI"
113     BADGE_SECRETARIA = "📄 Secretaria Acadêmica"
114     BADGE_ORCHESTRATOR = "Assistente IA"
115
116
117     def _sector_from_badge(agent: str) -> str:
118         if "TI" in agent:
119             return "ti"
120         if "Secretaria" in agent:
121             return "secretaria"
122         return ""
123
124
125     def _badge_from_sector(sector: str) -> str:
126         if sector == "ti":
127             return BADGE_TI
128         if sector == "secretaria":
129             return BADGE_SECRETARIA
130         return BADGE_ORCHESTRATOR
131
132
133     def _sector_from_member_response(response) -> str:
134         for member in reversed(response.member_responses or []):
135             name = (getattr(member, "agent_name", None) or "").lower()
136             agent_id = (getattr(member, "agent_id", None) or "").lower()
137             if "secretaria" in name or "secretaria" in agent_id:
138                 return "secretaria"
139             if "ti" in name or "ti" in agent_id:
140                 return "ti"
141             return "orchestrator"
142
143
144     def _agent_display_name(agent: Agent) -> str:
145         agent_id = (getattr(agent, "id", None) or "").lower()
146         if "secretaria" in agent_id or "secretaria" in agent.name.lower():
147             return BADGE_SECRETARIA
148         return BADGE_TI
149
150
```

```
151 def _active_sector_from_history(chat_history: List[Message]) -> str:
152     for message in reversed(chat_history):
153         if message.role == "bot":
154             sector = _sector_from_badge(message.agent)
155             if sector:
156                 return sector
157     return ""
158
159
160 def _resolve_fallback_agent(active_sector: str, chat_history:
    List[Message]) -> Agent:
161     if active_sector == "ti":
162         return agent_ti
163     if active_sector == "secretaria":
164         return agent_secretaria
165     last_sector = _active_sector_from_history(chat_history)
166     if last_sector == "ti":
167         return agent_ti
168     if last_sector == "secretaria":
169         return agent_secretaria
170     return agent_secretaria
171
172
173 TI_CHANGE_SIGNALS = (
174     "senha", "login", "wi-fi", "wifi", "vpn", "erro", "fora do ar",
175     "não consigo acessar", "nao consigo acessar", "não consigo entrar",
176     "nao consigo entrar",
177 )
178 SECRETARIA_CHANGE_SIGNALS = (
179     "matrícula", "matricula", "trancar", "trancamento", "documento",
180     "histórico escolar", "historico escolar", "diploma", "requerimento",
181     "cancelar matrícula", "cancelar matricula",
182 )
183
184
185 TECH_FAILURE_SIGNALS = (
186     "fora do ar", "indisponível", "indisponivel", "não funciona", "nao
    funciona",
187     "não abre", "nao abre", "erro 500", "erro 404", "erro técnico", "erro
    tecnico",
188     "instabilidade", "caiu o site", "caiu o sistema",
189 )
190
191
```

```
192 def _detect_ambiguous_mixed_case(user_message: str) -> Agent | None:
193     """Documento/burocracia + falha técnica na mesma mensagem → TI resolve primeiro."""
194     lower = user_message.lower()
195     has_admin = any(s in lower for s in SECRETARIA_CHANGE_SIGNALS)
196     has_tech = any(s in lower for s in TI_CHANGE_SIGNALS) or any(
197         s in lower for s in TECH_FAILURE_SIGNALS
198     )
199     if has_admin and has_tech:
200         return agent_ti
201     return None
202
203
204 def _detect_sector_change(user_message: str, active_sector: str) -> Agent | None:
205     """Corrige roteamento quando a nova mensagem indica mudança clara de setor."""
206     lower = user_message.lower()
207     if active_sector == "secretaria" and any(s in lower for s in TI_CHANGE_SIGNALS):
208         return agent_ti
209     if active_sector == "ti" and any(s in lower for s in SECRETARIA_CHANGE_SIGNALS):
210         return agent_secretaria
211     return None
212
213
214 def _build_contextual_message(chat_history: List[Message], user_message: str) -> str:
215     prior_messages = [m for m in chat_history[:-1] if m.role in ("user", "bot")][-4:]
216     if not prior_messages:
217         return user_message
218
219     transcript = []
220     for message in prior_messages:
221         speaker = "Aluno" if message.role == "user" else message.agent
222         transcript.append(f"{speaker}: {message.content[:800]}")
223
224     lower_new = user_message.lower()
225     change_hint = ""
226     if any(signal in lower_new for signal in TI_CHANGE_SIGNALS):
227         change_hint = (
```

```
228         "ATENÇÃO: a nova mensagem contém termos técnicos (login,  
229         senha, acesso, erro). "  
230     )  
231  
232     return (  
233         change_hint  
234         + "Use o histórico abaixo apenas como contexto. "  
235         "Responda somente à NOVA mensagem do aluno – o assunto pode ter  
236         mudado de setor.\n\n"  
237         + "\n\n".join(transcript)  
238         + f"\n\nNova mensagem do aluno: {user_message}"  
239     )  
240  
241     def _extract_team_reply(response) -> str:  
242         content = response.content  
243         if content and str(content).strip():  
244             return str(content)  
245  
246         for member in reversed(response.member_responses or []):  
247             member_content = getattr(member, "content", None)  
248             if member_content and str(member_content).strip():  
249                 return str(member_content)  
250  
251         return ""  
252  
253  
254     def _run_agent(agent: Agent, message: str, session_id: str) -> str:  
255         response = agent.run(message, session_id=session_id)  
256         return _extract_team_reply(response) or "Não foi possível obter uma  
257         resposta."  
258  
259     def _run_support_team(  
260         user_message: str,  
261         chat_history: List[Message],  
262         session_id: str,  
263         active_sector: str = "",  
264     ):  
265         contextual_message = _build_contextual_message(chat_history,  
266         user_message)  
267         ambiguous_agent = _detect_ambiguous_mixed_case(user_message)
```

```
268     if ambiguous_agent is not None:
269         bot_reply = _run_agent(ambiguous_agent, contextual_message,
270                                session_id)
271         return bot_reply, _agent_display_name(ambiguous_agent), "ti"
272
273     sector_change_agent = _detect_sector_change(user_message,
274                                                  active_sector)
275     if sector_change_agent is not None:
276         bot_reply = _run_agent(sector_change_agent, contextual_message,
277                                session_id)
278         sector = "ti" if sector_change_agent is agent_ti else "secretaria"
279         return bot_reply, _agent_display_name(sector_change_agent), sector
280
281     response = support_team.run(contextual_message, session_id=session_id)
282     bot_reply = _extract_team_reply(response)
283
284     if bot_reply.strip():
285         sector = _sector_from_member_response(response)
286         if sector == "orchestrator":
287             sector = active_sector or "orchestrator"
288         return bot_reply, _badge_from_sector(sector), sector
289
290     fallback_agent = _resolve_fallback_agent(active_sector, chat_history)
291     bot_reply = _run_agent(fallback_agent, contextual_message, session_id)
292     sector = "ti" if fallback_agent is agent_ti else "secretaria"
293     return bot_reply, _agent_display_name(fallback_agent), sector
294
295 WELCOME_MSG = Message(
296     role="bot",
297     content=(
298         "Olá! Bem-vindo ao **Portal de Atendimento Institucional do IFES**.\n\n"
299         "Nossa IA faz a triagem automática e encaminha sua solicitação para o setor correto:\n\n"
300         "💻 **TI** – senhas, sistemas, acesso, e-mail\n"
301         "📁 **Secretaria** – matrícula, documentos, trancamentos, histórico\n\n"
302         "Como posso ajudá-lo hoje?"
303     ),
304     agent="Sistema",
305     timestamp="",
```

```
306 # =====
307 # ESTADO DA APLICAÇÃO
308 # =====
309 class State(rx.State):
310     chat_history: List[Message] = [WELCOME_MSG]
311     user_input: str = ""
312     is_processing: bool = False
313     active_sector: str = ""
314     session_id: str = rx.LocalStorage(name="session_id")
315     chat_history_json: str = rx.LocalStorage(name="ifes_chat_history_v2")
316     _history_restored: bool = False
317
318     def on_load(self):
319         self._restore_from_storage()
320         if not self.session_id:
321             self.session_id = str(uuid.uuid4())
322
323     def sync_from_storage(self):
324         """Restaura histórico após o LocalStorage sincronizar com o
325         backend."""
326         self._restore_from_storage()
327         if not self.session_id:
328             self.session_id = str(uuid.uuid4())
329
330     def _restore_from_storage(self):
331         if self._history_restored or not self.chat_history_json:
332             return
333         try:
334             data = json.loads(self.chat_history_json)
335             if isinstance(data, list) and len(data) > 0:
336                 self.chat_history = [Message(**item) for item in data]
337                 self.active_sector =
338                     _active_sector_from_history(self.chat_history)
339                 self._history_restored = True
340         except Exception:
341             self.chat_history = [WELCOME_MSG]
342             self.active_sector = ""
343
344     def _save_history(self):
345         try:
346             self.chat_history_json = json.dumps(
347                 [
348                     {
349                         "role": m.role,
```

```
348         "content": m.content,
349         "agent": m.agent,
350         "timestamp": m.timestamp,
351     }
352     for m in self.chat_history
353 ]
354 )
355 except Exception:
356     pass
357
358 def set_input(self, value: str):
359     self.user_input = value
360
361 def clear_chat(self):
362     self.session_id = str(uuid.uuid4())
363     self.active_sector = ""
364     self._history_restored = False
365     self.chat_history = [
366         Message(
367             role="bot",
368             content=WELCOME_MSG.content,
369             agent="Sistema",
370             timestamp=datetime.now().strftime("%H:%M"),
371         )
372     ]
373     self.chat_history_json = ""
374     self._save_history()
375
376 async def handle_submit(self):
377     user_message = self.user_input.strip()
378     if not user_message or self.is_processing:
379         return
380
381     now = datetime.now().strftime("%H:%M")
382
383     # Adiciona mensagem do usuário
384     self.chat_history.append(
385         Message(role="user", content=user_message, agent="Você",
386             timestamp=now)
387     )
388     self._save_history()
389     self.user_input = ""
390     self.is_processing = True
391     yield
```

```
391
392     # =====
393     # DETECÇÃO DE PERGUNTAS SOBRE O HISTÓRICO (META)
394     # =====
395     meta_keywords = [
396         "última dúvida", "ultima duvida", "o que eu perguntei",
397         "qual foi minha", "minhas perguntas anteriores", "histórico da
398         conversa",
399         "o que eu falei", "última mensagem", "contexto anterior"
400     ]
401     is_meta = any(kw in user_message.lower() for kw in meta_keywords)
402
403     if is_meta:
404         ultimas = [m for m in self.chat_history if m.role == "user"]
405         [-5:]
406         if ultimas:
407             conteudo = "Aqui estão suas últimas interações:\n\n"
408             for i, msg in enumerate(ultimas, 1):
409                 conteudo += f"***{i}.* {msg.content}\n"
410         else:
411             conteudo = "Ainda não temos interações anteriores
412             registradas nesta conversa."
413
414     self.chat_history.append(
415         Message(role="bot", content=conteudo, agent="Assistente
416         IA", timestamp=now)
417     )
418     self._save_history()
419     self.is_processing = False
420     return
421
422     # =====
423
424     # --- Fluxo normal: Team do Agno como roteador principal ---
425     try:
426         bot_reply, agent_name, new_sector = _run_support_team(
427             user_message,
428             self.chat_history,
429             self.session_id,
430             self.active_sector,
431         )
432         if new_sector in ("ti", "secretaria"):
433             self.active_sector = new_sector
434
435     except Exception as e:
```



```
431         bot_reply = f"***Erro interno:** {str(e)}"
432         agent_name = "Sistema"
433
434         # Adiciona resposta do bot
435         self.chat_history.append(
436             Message(role="bot", content=bot_reply, agent=agent_name,
437                     timestamp=datetime.now().strftime("%H:%M"))
438         )
439         self._save_history()
440         self.is_processing = False
441         # =====
442         # COMPONENTES VISUAIS (mantidos iguais)
443         # =====
444         def agent_badge(agent: str) -> rx.Component:
445             return rx.box(
446                 rx.text(agent, size="1", weight="bold"),
447                 bg=rx.cond(
448                     agent.contains("TI"),
449                     "#e3f2fd",
450                     rx.cond(agent.contains("Secretaria"), "#e0f2f1", "#eceff1"),
451                 ),
452                 color=rx.cond(
453                     agent.contains("TI"),
454                     "#1565c0",
455                     rx.cond(agent.contains("Secretaria"), "#00695c", "#37474f"),
456                 ),
457                 border_radius="20px",
458                 padding_x="10px",
459                 padding_y="3px",
460                 display="inline-block",
461                 margin_bottom="6px",
462             )
463
464         def message_bubble(message: Message) -> rx.Component:
465             is_user = message.role == "user"
466             return rx.box(
467                 rx.vstack(
468                     rx.cond(
469                         ~is_user,
470                         rx.hstack(
471                             agent_badge(message.agent),
472                             rx.text(message.timestamp, size="1", color="#90a4ae"),
473                             spacing="2",
```

```
474         align_items="center",
475     ),
476     rx.hstack(
477         rx.text("Você", size="1", weight="bold",
478             color="#5c6bc0"),
479         rx.text(message.timestamp, size="1", color="#90a4ae"),
480         spacing="2",
481         justify="end",
482         width="100%",
483     ),
484     rx.box(
485         rx.markdown(message.content),
486         bg=rx.cond(
487             is_user,
488             "linear-gradient(135deg, #3949ab, #1a237e)",
489             "#ffffff",
490         ),
491         color=rx.cond(is_user, "#ffffff", "#263238"),
492         padding="14px 18px",
493         border_radius=rx.cond(
494             is_user,
495             "18px 18px 4px 18px",
496             "18px 18px 18px 4px",
497         ),
498         box_shadow=rx.cond(
499             is_user,
500             "0 2px 12px rgba(57,73,171,0.25)",
501             "0 2px 8px rgba(0,0,0,0.07)",
502         ),
503         border=rx.cond(is_user, "none", "1px solid #e8eaf6"),
504         width="100%",
505     ),
506     spacing="1",
507     align_items=rx.cond(is_user, "flex-end", "flex-start"),
508     width="100%",
509 ),
510 max_width="78%",
511 align_self=rx.cond(is_user, "flex-end", "flex-start"),
512 padding_x="4px",
513 class_name="message-item",
514 )
515
516 def typing_indicator() -> rx.Component:
```

```
517     return rx.cond(  
518         State.is_processing,  
519         rx.box(  
520             rx.hstack(  
521                 rx.box(width="8px", height="8px", border_radius="50%",  
522                     bg="#3949ab", class_name="dot dot1"),  
523                 rx.box(width="8px", height="8px", border_radius="50%",  
524                     bg="#3949ab", class_name="dot dot2"),  
525                 rx.box(width="8px", height="8px", border_radius="50%",  
526                     bg="#3949ab", class_name="dot dot3"),  
527                 spacing="1",  
528                 align_items="center",  
529             ),  
530             bg="white",  
531             padding="12px 16px",  
532             border_radius="18px 18px 18px 4px",  
533             border="1px solid #e8eaf6",  
534             box_shadow="0 2px 8px rgba(0,0,0,0.07)",  
535         ),  
536         align_self="flex-start",  
537         padding_x="4px",  
538     ),  
539     rx.box(),  
540 )  
541  
542 def chat_window() -> rx.Component:  
543     return rx.box(  
544         rx.vstack(  
545             rx.foreach(State.chat_history, message_bubble),  
546             typing_indicator(),  
547             spacing="4",  
548             width="100%",  
549             padding_bottom="8px",  
550         ),  
551         id="chat-scroll",  
552         flex="1",  
553         overflow_y="auto",  
554         width="100%",  
555         padding="24px 20px",  
556         bg="#f4f6fb",  
557         class_name="chat-window",  
558     )  
559 )
```

```
558 def header() -> rx.Component:
559     return rx.box(
560         rx.hstack(
561             rx.box(
562                 rx.text("IFES", size="3", weight="bold", color="white"),
563                 bg="rgba(255,255,255,0.15)",
564                 border_radius="10px",
565                 padding="8px 12px",
566                 border="1px solid rgba(255,255,255,0.2)",
567             ),
568             rx.vstack(
569                 rx.text("Portal de Atendimento Institucional", size="4",
570                     weight="bold", color="white", line_height="1"),
571                 rx.text("Bot de Suporte com Roteamento Inteligente por
572                     IA", size="1", color="rgba(255,255,255,0.75)",
573                     spacing="1",
574                     align_items="flex-start",
575                 ),
576                 rx.spacer(),
577                 rx.hstack(
578                     rx.box(width="8px", height="8px", border_radius="50%",
579                         bg="#69f0ae", box_shadow="0 0 6px #69f0ae"),
580                     rx.text("Online", size="1",
581                         color="rgba(255,255,255,0.85)", weight="medium"),
582                     spacing="1",
583                     align_items="center",
584                 ),
585                 rx.button(
586                     rx.hstack(rx.icon("printer", size=14), rx.text("Imprimir",
587                         size="1"), spacing="1"),
588                     on_click=rx.call_script("window.print()"),
589                     bg="rgba(255,255,255,0.12)", color="white", border="1px
590                     solid rgba(255,255,255,0.25)",
591                     border_radius="8px", padding_x="14px", padding_y="8px",
592                     cursor="pointer",
593                     _hover={"bg": "rgba(255,255,255,0.22)"}, class_name="no-
594                     print",
595                 ),
596                 rx.button(
597                     rx.hstack(rx.icon("rotate-ccw", size=14), rx.text("Nova
598                         Conversa", size="1"), spacing="1"),
599                     on_click=State.clear_chat,
600                     bg="rgba(255,255,255,0.12)", color="white", border="1px
601                     solid rgba(255,255,255,0.25)",
```

```
592         border_radius="8px", padding_x="14px", padding_y="8px",
593         cursor="pointer",
594         _hover={"bg": "rgba(255,255,255,0.22)"}, class_name="no-
595         print",
596     ),
597     spacing="3", align_items="center", width="100%",
598     ),
599     bg="linear-gradient(135deg, #1a237e 0%, #283593 50%, #3949ab
600     100%)",
601     padding="18px 24px", width="100%", class_name="portal-header",
602 )
603
604 def sector_chips() -> rx.Component:
605     chip_style = dict(border_radius="20px", padding_x="14px",
606     padding_y="7px", cursor="pointer", size="1", font_size="12px")
607     return rx.hstack(
608         rx.text("Atalhos:", size="1", color="#78909c", weight="medium"),
609         rx.button("🔄 Resetar senha", on_click=State.set_input("Preciso
610         resetar minha senha do portal acadêmico"),
611         bg="#e8eaf6", color="#3949ab", border="none",
612         _hover={"bg": "#c5cae9"}, **chip_style),
613         rx.button("🔒 Trancar disciplina",
614         on_click=State.set_input("Quero trancar uma disciplina"),
615         bg="#e0f2f1", color="#00695c", border="none",
616         _hover={"bg": "#b2dfdb"}, **chip_style),
617         rx.button("📄 Solicitar histórico",
618         on_click=State.set_input("Preciso solicitar meu histórico
619         escolar"),
620         bg="#e0f2f1", color="#00695c", border="none",
621         _hover={"bg": "#b2dfdb"}, **chip_style),
622         rx.button("🔑 Problema de acesso", on_click=State.set_input("Não
623         consigo acessar o sistema Q-Acadêmico"),
624         bg="#e8eaf6", color="#3949ab", border="none",
625         _hover={"bg": "#c5cae9"}, **chip_style),
626         spacing="2", align_items="center", wrap="wrap", padding_x="20px",
627         padding_y="10px",
628         bg="#fafbff", border_bottom="1px solid #e8eaf6", class_name="no-
629         print",
630     )
631
632 def input_area() -> rx.Component:
633     return rx.box(
634         rx.hstack(
635             rx.input(
```

```
621         value=State.user_input, on_change=State.set_input,
622         placeholder="Descreva sua solicitação... (ex: esqueci
623         minha senha, quero trancar matrícula)",
624         width="100%", disabled=State.is_processing, size="3",
625         border_radius="12px",
626         border="1.5px solid #c5cae9", padding_x="16px",
627         bg="white", color="#263238",
628         _focus={"border_color": "#3949ab", "box_shadow": "0 0 0
629         3px rgba(57,73,171,0.12)", "outline": "none"},
630         _placeholder={"color": "#b0bec5"},
631     ),
632     rx.button(
633         rx.cond(
634             State.is_processing,
635             rx.hstack(rx.spinner(size="2", color="white"),
636             rx.text("Processando...", size="2", spacing="2"),
637             rx.hstack(rx.icon("send", size=16), rx.text("Enviar",
638             size="2", weight="medium"), spacing="2"),
639         ),
640         on_click=State.handle_submit,
641         bg=rx.cond(State.is_processing, "#7986cb", "#3949ab"),
642         color="white", border_radius="12px", padding_x="22px",
643         padding_y="0", height="44px",
644         disabled=State.is_processing,
645         cursor=rx.cond(State.is_processing, "not-allowed",
646         "pointer"),
647         box_shadow="0 2px 8px rgba(57,73,171,0.3)",
648         _hover={"bg": "#1a237e", "box_shadow": "0 4px 12px
649         rgba(57,73,171,0.4)"},
650         transition="all 0.2s ease",
651     ),
652     spacing="3", align_items="center", width="100%",
653 ),
654 rx.hstack(
655     rx.icon("info", size=12, color="#90a4ae"),
656     rx.text("Clique em Enviar • A IA identifica e encaminha para
657     TI ou Secretaria automaticamente",
658     size="1", color="#90a4ae"),
659     spacing="1", align_items="center", margin_top="8px",
660 ),
661 padding="16px 20px 18px", bg="white", border_top="1px solid
662 #e8eaf6", width="100%", class_name="no-print",
663 )
664 )
665 )
```

```
653 def footer_info() -> rx.Component:
654     return rx.hstack(
655         rx.hstack(rx.box(width="6px", height="6px", border_radius="50%",
656             bg="#69f0ae"),
657             rx.text("TI • Infraestrutura e Sistemas", size="1",
658                 color="#78909c"), spacing="1"),
659         rx.text(".", color="#cfd8dc", size="1"),
660         rx.hstack(rx.box(width="6px", height="6px", border_radius="50%",
661             bg="#4db6ac"),
662             rx.text("Secretaria Acadêmica", size="1",
663                 color="#78909c"), spacing="1"),
664         rx.spacer(),
665         rx.text("Sessão ativa", size="1", color="#b0bec5"),
666         spacing="2", align_items="center", padding_x="20px",
667         padding_y="8px",
668         bg="#f4f6fb", border_top="1px solid #e8eaf6", width="100%",
669         class_name="no-print",
670     )
671
672 def index() -> rx.Component:
673     return rx.box(
674         rx.html("""<style>
675 @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;
676 600;700&display=swap');
677 * { font-family: 'Inter', system-ui, sans-serif !important; box-sizing:
678 border-box; }
679 body { margin: 0; padding: 0; background: #e8eaf6; }
680 .chat-window::-webkit-scrollbar { width: 5px; }
681 .chat-window::-webkit-scrollbar-track { background: transparent; }
682 .chat-window::-webkit-scrollbar-thumb { background: #c5cae9; border-
683 radius: 10px; }
684 .chat-window::-webkit-scrollbar-thumb:hover { background: #9fa8da; }
685 @keyframes bounce { 0%, 80%, 100% { transform: scale(0.6); opacity: 0.4; }
686 40% { transform: scale(1); opacity: 1; } }
687 .dot { animation: bounce 1.4s infinite ease-in-out; }
688 .dot1 { animation-delay: 0s; } .dot2 { animation-delay: 0.2s; } .dot3
689 { animation-delay: 0.4s; }
690 #chat-scroll { scroll-behavior: smooth; }
691 .message-item { animation: fadeSlide 0.25s ease-out; }
692 @keyframes fadeSlide { from { opacity: 0; transform: translateY(8px); } to
693 { opacity: 1; transform: translateY(0); } }
694 @media print {
695     @page { size: A4 portrait; margin: 15mm 20mm; }
696     body { background: white !important; }
```

```
685 .no-print { display: none !important; }
686 .portal-header { background: #1a237e !important; print-color-adjust:
exact; padding: 12px 20px !important; }
687 .chat-window { height: auto !important; max-height: none !important;
overflow: visible !important; background: white !important; }
688 .message-item { break-inside: avoid; }
689 * { print-color-adjust: exact; -webkit-print-color-adjust: exact; }
690 }
691 </style>
692 <script>
693 const observer = new MutationObserver(() => {
694   const el = document.getElementById('chat-scroll');
695   if (el) el.scrollTop = el.scrollHeight;
696 });
697 document.addEventListener('DOMContentLoaded', () => {
698   const el = document.getElementById('chat-scroll');
699   if (el) {
700     observer.observe(el, { childList: true, subtree: true });
701     el.scrollTop = el.scrollHeight;
702   }
703   setTimeout(() => {
704     const btn = document.getElementById('sync-storage-trigger');
705     if (btn) btn.click();
706   }, 300);
707 });
708 </script>"""),
709     rx.vstack(
710       header(),
711       sector_chips(),
712       chat_window(),
713       input_area(),
714       footer_info(),
715       spacing="0", width="100%", height="100%",
border_radius="16px",
716       overflow="hidden", box_shadow="0 8px 40px
rgba(26,35,126,0.18)",
717     ),
718     rx.button(
719       id="sync-storage-trigger",
720       on_click=State.sync_from_storage,
721       display="none",
722     ),
723     width="min(95vw, 920px)", height="min(95vh, 820px)",
margin="auto",
```



```
724         position="fixed", top="50%", left="50%",  
725         transform="translate(-50%, -50%)",  
726     )  
727 app = rx.App(style={"background": "#e8eaf6", "min_height": "100vh"})  
728 app.add_page(index, title="Portal de Atendimento IFES",  
              on_load=State.on_load)
```

12.2 testes/testar_roteamento.py

```
1  """Valida os 10 casos de roteamento pelo fluxo do  
   projeto_ia_roteamento.py."""  
2  import sys  
3  import time  
4  import uuid  
5  from pathlib import Path  
6  
7  ROOT = Path(__file__).resolve().parent.parent  
8  sys.path.insert(0, str(ROOT))  
9  
10 from projeto_ia_roteamento.projeto_ia_roteamento import ( # noqa: E402  
11     Message,  
12     WELCOME_MSG,  
13     _active_sector_from_history,  
14     _run_support_team,  
15 )  
16  
17 CASES = [  
18     ("01", "Minha senha do portal do aluno não funciona.", "ti"),  
19     ("02", "Quero cancelar minha matrícula, como faço?", "secretaria"),  
20     ("03", "Preciso do diploma, mas o site de emissão está fora do ar.",  
21         "ti"),  
21     ("04", "Não consigo acessar a rede Wi-Fi da biblioteca.", "ti"),  
22     ("05", "Qual o prazo para solicitar aproveitamento de disciplinas?",  
23         "secretaria"),  
23     ("06", "Meu nome está escrito errado no portal, preciso arrumar.",  
24         "secretaria"),  
24     ("07", "Estou tentando enviar meu TCC pelo sistema, mas dá erro 500.",  
25         "ti"),  
25     ("08", "Onde pego o comprovante de matrícula para o estágio?",  
26         "secretaria"),  
26     ("10", "Olá, boa tarde! Gostaria de ajuda.", "orchestrator"),
```

```
27 ]
28
29 FOLLOW_UP = (
30     "09",
31     "E se eu esquecer o meu login na hora de baixar?",
32     "ti",
33     "08",
34 )
35
36
37 def _run_case(case_id: str, prompt: str, expected: str, session_id: str,
38             history: list):
39     history.append(Message(role="user", content=prompt, agent="Você",
40                          timestamp="00:00"))
41     active_sector = _active_sector_from_history(history[:-1])
42
43     for attempt in range(3):
44         try:
45             _, badge, sector = _run_support_team(prompt, history,
46                                                  session_id, active_sector)
47             break
48         except Exception as exc:
49             if "429" in str(exc) and attempt < 2:
50                 time.sleep(65)
51                 continue
52             raise
53
54     history.append(Message(role="bot", content="...", agent=badge,
55                          timestamp="00:01"))
56     time.sleep(5)
57
58     ok = sector == expected
59     status = "OK" if ok else "FALHOU"
60     print(f" [{status}] {case_id}: esperado={expected}, obtido={sector} ({badge})")
61     return ok
62
63 def main():
64     print("=" * 60)
65     print("Testes de roteamento – projeto_ia_roteamento.py")
66     print("=" * 60)
67
68     results = []
```

```
66
67     for case_id, prompt, expected in CASES:
68         session_id = str(uuid.uuid4())
69         history = [WELCOME_MSG]
70         ok = _run_case(case_id, prompt, expected, session_id, history)
71         results.append(ok)
72
73     session_id = str(uuid.uuid4())
74     history = [WELCOME_MSG]
75     parent_id = FOLLOW_UP[3]
76     case_id, prompt, expected = CASES[7]
77     _run_case(case_id, prompt, expected, session_id, history)
78     ok = _run_case(FOLLOW_UP[0], FOLLOW_UP[1], FOLLOW_UP[2], session_id,
79                   history)
79     results.append(ok)
80     print(f" (caso {FOLLOW_UP[0]} continua sessão do caso {parent_id})")
81
82     passed = sum(results)
83     total = len(results)
84     print("-" * 60)
85     print(f"Resultado: {passed}/{total} ({100 * passed // total}%)")
86     sys.exit(0 if passed == total else 1)
87
88
89 if __name__ == "__main__":
90     main()
```

12.3 testes/salvar_traces.py

```
1     """Captura reasoning traces do MVP projeto_ia_roteamento.py para o
   relatório."""
2     import sys
3     import time
4     import uuid
5     from contextlib import contextmanager
6     from datetime import datetime
7     from pathlib import Path
8
9     ROOT = Path(__file__).resolve().parent.parent
10    sys.path.insert(0, str(ROOT))
11
12    from projeto_ia_roteamento.projeto_ia_roteamento import ( # noqa: E402
```

```
13     Message,  
14     WELCOME_MSG,  
15     _active_sector_from_history,  
16     _run_support_team,  
17     agent_secretaria,  
18     agent_ti,  
19     support_team,  
20 )  
21  
22 TRACES_DIR = Path(__file__).resolve().parent / "traces" /  
23     "projeto_ia_roteamento"  
24  
25 CASES = [  
26     ("01", "trace_01_ti.txt", "Minha senha do portal do aluno não  
27     funciona.", "ti"),  
28     ("02", "trace_02_secretaria.txt", "Quero cancelar minha matrícula,  
29     como faço?", "secretaria"),  
30     ("03", "trace_03_ti_ambiguo.txt", "Preciso do diploma, mas o site de  
31     emissão está fora do ar.", "ti"),  
32     ("04", "trace_04_ti.txt", "Não consigo acessar a rede Wi-Fi da  
33     biblioteca.", "ti"),  
34     ("05", "trace_05_secretaria.txt", "Qual o prazo para solicitar  
35     aproveitamento de disciplinas?", "secretaria"),  
36     ("06", "trace_06_secretaria_ambiguo.txt", "Meu nome está escrito  
37     errado no portal, preciso arrumar.", "secretaria"),  
38     ("07", "trace_07_ti.txt", "Estou tentando enviar meu TCC pelo sistema,  
39     mas dá erro 500.", "ti"),  
40     ("08", "trace_08_secretaria.txt", "Onde pego o comprovante de  
41     matrícula para o estágio?", "secretaria"),  
42     ("10", "trace_10_triagem_orquestrador.txt", "Olá, boa tarde! Gostaria  
43     de ajuda.", "orchestrator"),  
44 ]  
45  
46 CASE_09 = (  
47     "09",  
48     "trace_09_correcao_rota_ti.txt",  
49     "E se eu esquecer o meu login na hora de baixar?",  
50     "ti",  
51 )  
52  
53 @contextmanager  
54 def _capture_output(path: Path):  
55     path.parent.mkdir(parents=True, exist_ok=True)
```

```
47     with path.open("w", encoding="utf-8") as handle:
48         old_stdout = sys.stdout
49         old_stderr = sys.stderr
50         sys.stdout = handle
51         sys.stderr = handle
52         try:
53             yield handle
54         finally:
55             sys.stdout = old_stdout
56             sys.stderr = old_stderr
57
58
59 def _enable_debug():
60     support_team.debug_mode = True
61     agent_ti.debug_mode = True
62     agent_secretaria.debug_mode = True
63
64
65 def _write_header(handle, case_id: str, prompt: str, expected: str, extra:
66 str = ""):
67     now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
68     handle.write("# MVP – projeto_ia_roteamento.py\n")
69     handle.write(f"# Caso ID: {case_id}\n")
70     handle.write(f"# Prompt: \"{prompt}\"\n")
71     handle.write(f"# Destino esperado: {expected.upper()}\n")
72     if extra:
73         handle.write(f"# {extra}\n")
74     handle.write(f"# Executado em: {now}\n")
75     handle.write("#" + "=" * 59 + "\n\n")
76
77 def _run_and_save(
78     case_id: str,
79     filename: str,
80     prompt: str,
81     expected: str,
82     session_id: str,
83     history: list,
84     extra: str = "",
85 ):
86     history.append(Message(role="user", content=prompt, agent="Você",
87 timestamp="00:00"))
88     active_sector = _active_sector_from_history(history[:-1])
89     out_path = TRACES_DIR / filename
```

```
89
90     for attempt in range(3):
91         try:
92             with _capture_output(out_path) as handle:
93                 _write_header(handle, case_id, prompt, expected, extra)
94                 handle.flush()
95                 old_stdout = sys.stdout
96                 old_stderr = sys.stderr
97                 sys.stdout = handle
98                 sys.stderr = handle
99                 try:
100                     print(f"--- Execução caso {case_id} ---")
101                     print(f"session_id: {session_id}")
102                     print(f"active_sector: {active_sector or '(nenhum)'}")
103                     print(f"prompt: {prompt}\n")
104
105                     bot_reply, badge, sector = _run_support_team(
106                         prompt, history, session_id, active_sector
107                     )
108
109                     print(f"\n--- Resultado ---")
110                     print(f"badge: {badge}")
111                     print(f"sector: {sector}")
112                     print(f"resposta (início): {bot_reply[:300]}...")
113                 finally:
114                     sys.stdout = old_stdout
115                     sys.stderr = old_stderr
116                 break
117         except Exception as exc:
118             if "429" in str(exc) and attempt < 2:
119                 time.sleep(65)
120                 continue
121             raise
122
123     history.append(Message(role="bot", content="...", agent=badge,
124                             timestamp="00:01"))
125     ok = sector == expected
126     print(f"  [{'OK' if ok else 'FALHOU'}] {case_id} -> {out_path.name}
127           (sector={sector})")
128     return ok
129
130 def main():
131     print("=" * 60)
```

```
131     print("Captura de traces – projeto_ia_roteamento.py")
132     print(f"Destino: {TRACES_DIR}")
133     print("=" * 60)
134
135     _enable_debug()
136     TRACES_DIR.mkdir(parents=True, exist_ok=True)
137
138     for case_id, filename, prompt, expected in CASES:
139         session_id = str(uuid.uuid4())
140         history = [WELCOME_MSG]
141         _run_and_save(case_id, filename, prompt, expected, session_id,
142                       history)
143         time.sleep(5)
144
145         session_id = str(uuid.uuid4())
146         history = [WELCOME_MSG]
147         c08 = CASES[7]
148         _run_and_save(c08[0], c08[1], c08[2], c08[3], session_id, history)
149         time.sleep(5)
150
151         c09 = CASE_09
152         _run_and_save(
153             c09[0],
154             c09[1],
155             c09[2],
156             c09[3],
157             session_id,
158             history,
159             extra="Continuação do caso 08 (mesma sessão)",
160         )
161
162     print("-" * 60)
163     print(f"Traces salvos em: {TRACES_DIR}")
164     print("Próximo passo: atualizar docs/relatorio_tecnico.md com as análises.")
165
166 if __name__ == "__main__":
167     main()
```